

AutoCare AI – Vehicle Service & Maintenance SaaS (V1)

Comprehensive Workplan & Technical Specification Document

1. Project Title

AutoCare AI – Vehicle Service & Maintenance SaaS (V1)

2. Refined Problem Statement

Automotive workshops require an integrated digital platform to manage customer profiles, maintain historical vehicle records, schedule service bays, and deliver AI-assisted diagnostic recommendations. AutoCare AI (V1) is a multi-tenant SaaS application that allows workshops to onboard customers and vehicles, receive problem reports, generate AI-powered diagnostic insights, coordinate conflict-free service appointments, and generate itemized billing. The platform guarantees strict multi-tenant data isolation and enforces transactional appointment concurrency control.

3. Project Overview

AutoCare AI V1 is a multi-tenant web platform for automotive maintenance management. Each workshop (tenant) operates within an isolated workspace to manage staff (mechanics/admins) and customer accounts, maintain vehicle inventories, log customer issues, generate diagnostic reports via an integrated LLM, schedule service appointments, track parts and labor costs, and communicate via appointment-specific messaging threads. The customer-facing web interface (built with Next.js) allows vehicle owners to monitor vehicle health, review open diagnostic reports, schedule appointments, review maintenance logs, and inspect invoices.

4. Revised Solution

- **Multi-Tenant SaaS:** Workshops operate in logically isolated workspaces. Customers register and link to a workshop using a unique workshop access code. Multi-tenancy is anchored to the root user profile and enforced across all dependent entities without redundant key duplication.
- **Diagnostic & Service Pipeline:** A customer logs an issue report; the backend performs keyword retrieval against diagnostic procedures and queries an LLM to generate a structured SOLUTION_REPORT with linked keywords stored in SOLUTION_KEYWORD. The workshop reviews the solution report and books a service appointment. Upon completion, the mechanic logs work details, parts cost, and labor cost directly in the APPOINTMENT record, which triggers the generation of an INVOICE.
- **Direct Communication:** Customers and mechanics communicate through an in-app chat tied directly to each appointment, implemented via a unified conversation message log.
- **Predictive Maintenance Reminders:** The system creates rule-based reminders in the REMINDER table based on vehicle mileage thresholds and elapsed calendar intervals.

5. Project Goals & Objectives

- Deliver a fully functional MVP supporting all nine core hero features.
- Implement a 3NF-compliant relational database schema within a strict 12-table budget.
- Demonstrate advanced relational concepts: EER supertype/subtype specialization, transactional concurrency locking, and relational multi-tenancy.

- Integrate a retrieval-augmented LLM diagnostic pipeline with structured output persistence.
- Implement an event-driven modular backend architecture using Node.js, Express.js (with raw SQL via mysql2 driver), and Next.js.

6. Hero Features (9)

- 1. Workshop Dashboard (Tenant Management):** Workshop administrators manage shop profiles, staff accounts, customer rosters, and high-level shop metrics within their isolated tenant partition.
- 2. Role-Based Access & Onboarding:** Secure authentication for Admins, Mechanics, and Customers. Customers join via workshop access codes; Admins create Mechanic accounts with unique employee codes.
- 3. Vehicle Health Dashboard:** Customers inspect vehicle profiles (make, model, year, VIN, odometer), historical service logs, active maintenance reminders, and open problem reports.
- 4. AI-Assisted Diagnostic Assistant:** Customer-submitted symptoms trigger procedural context retrieval and LLM processing to produce a SOLUTION_REPORT containing probable causes, recommended actions, urgency levels, and confidence scores, indexed via SOLUTION_KEYWORD entries.
- 5. Appointment Scheduling (Conflict-Free):** Service booking engine that prevents overlapping appointments for the same mechanic using transactional range checks.
- 6. Digital Service & Maintenance History:** Detailed logs of completed maintenance capturing work descriptions, parts costs, and labor costs directly within appointment records, viewable chronologically.
- 7. Customer–Mechanic Communication:** Appointment-specific communication stream stored in a single CONVERSATION table where each record represents an individual message.
- 8. Simple Invoicing & Payment Tracking:** Automatic generation of an INVOICE upon service completion to track total billing amounts and settlement status (Pending/Paid).
- 9. Maintenance Reminders & Alerts:** Rule-based predictive notifications stored in the REMINDER table and triggered by mileage or calendar intervals.

7. User Roles / Actors

- **Workshop Admin:** Manages workshop configuration, registers mechanic staff, and monitors operations.
- **Mechanic / Staff:** Reviews assigned repair tickets, examines AI solution reports, records parts/labor costs upon completion, and chats with customers.
- **Customer:** Registers via workshop access code, registers vehicles, submits problem reports, books appointments, and reviews service history and invoices.
- **System / AI Diagnostic Bot:** Background service that executes retrieval and generates structured diagnostic solution records.

8. V1 Scope (Included)

- Single-workshop onboarding and strict relational multi-tenant isolation.
- Modular monolithic architecture: Next.js frontend, Express.js (Node.js) backend with raw SQL driver (mysql2), MySQL 8.0 database.
- 3NF schema containing exactly 11 core tables.
- Google Gemini REST API integration for structured diagnostic report synthesis.
- In-process Node.js asynchronous event dispatching for system alerts.
- Containerized local development using Docker Compose and automated CI via GitHub Actions.

9. Removed / Deferred Complexity

- No public multi-workshop marketplace or SaaS billing tiers.
- No external vector database (retrieval uses internal keyword matching).
- No multi-currency accounting engine or external payment gateway integrations (Stripe/PayPal deferred).
- No third-party OAuth/SSO or MFA mechanisms (standard JWT authentication).
- No distributed event broker (Kafka/RabbitMQ deferred; in-process Node.js events used).

10. Functional Requirements

- **Tenant Registration:** Workshop creates an access code for customer registration.
- **User Onboarding:** Users register as Customers via workshop access codes or are provisioned by Admins as Mechanics with employee codes.
- **Vehicle Registration:** Customers link multiple vehicles to their account.
- **Problem Reporting:** Customers submit symptom descriptions for a registered vehicle.
- **Diagnostic Generation:** Backend retrieves matching procedures, calls the LLM, and persists a structured SOLUTION_REPORT and associated SOLUTION_KEYWORD rows.
- **Appointment Scheduling:** Customers book appointments; the system transactionally validates that the assigned mechanic has no overlapping time ranges.
- **Service Completion:** Mechanics record work descriptions, parts cost, and labor cost on the APPOINTMENT record.
- **Invoicing:** Completing an appointment generates an INVOICE with the computed total amount.
- **Messaging:** Customers and mechanics exchange messages within an appointment chat thread stored in CONVERSATION.
- **Maintenance Reminders:** Automated engine schedules predictive service reminders in REMINDER.

11. Non-Functional Requirements

- **Security:** Stateless JWT authentication, BCrypt password hashing, role-based authorization, and strict multi-tenant boundary checks.
- **Reliability & Concurrency:** ACID transactions and pessimistic locking on appointment range queries to prevent mechanic double-booking.
- **Performance:** Lightweight schema indexes on all foreign keys, unique natural keys, and search keywords.
- **Maintainability:** Layered architecture (Routes/Controllers, Services/Business Logic, Repositories/Data Access Layer with parameterized Raw SQL) with normalized schema models and zero ORM abstractions.

12. Simplified System Architecture



13. Complete Technology Stack

- **Frontend:** Next.js (App Router), React, TypeScript, Tailwind CSS.
- **Backend:** Node.js, Express.js, Raw SQL (mysql2 driver with connection pooling & transactions), jsonwebtoken (Stateless JWT), bcryptjs.
- **Database:** MySQL 8.0.
- **AI Engine:** Google Gemini REST API.
- **Containerization & CI/CD:** Docker, Docker Compose, GitHub Actions.

14. Technology Priority

- **Core (Mandatory):** Express.js backend (Raw SQL / mysql2), MySQL relational schema, Next.js interface, JWT security, Gemini AI diagnostic workflow, transactional booking lock.
- **Secondary:** Docker Compose setup, Express health check endpoints, Tailwind UI styling.
- **Deferred:** Redis caching, WebSockets, external payment gateways, distributed tracing.

15. Database-First Development Strategy

- Map all domain requirements into clean relational entities with surrogate primary keys.
- Normalize all structures to 3NF, eliminating partial and transitive dependencies.
- Implement database schema DDL with explicit foreign key constraints and cascade actions.
- Build raw SQL repository data access modules (parameterized queries, transaction helpers) matching database tables and configure automated database integration tests.

16. ER/EER Design Preparation

- **Surrogate Keys:** Standardize on integer auto-increment primary keys across all parent tables.
- **Role Specialization:** Implement an EER supertype/subtype hierarchy where USER is the supertype and CUSTOMER and MECHANIC are subtype tables holding role-specific attributes.
- **Atomic Decompositions:** Multi-valued symptom keywords are normalized into SOLUTION_KEYWORD (1NF).
- **Elimination of Transitive Dependencies:** Child entities reference their direct parent only; tenant ownership is derived via the parent user's workshop_id (3NF).

17. Target Entity / Table Budget

The final normalized architecture consists of 11 tables (satisfying the ≤ 12 table budget):

1. WORKSHOP (Tenant organization)
2. USER (Supertype authentication and profile entity)
3. CUSTOMER (Subtype table for customer-specific data)
4. MECHANIC (Subtype table for mechanic-specific data)
5. VEHICLE (Customer-owned vehicles)
6. REMINDER (Predictive maintenance reminders)
7. PROBLEM_REPORT (Customer-reported issues)
8. SOLUTION_REPORT (AI-generated diagnostic solution reports)
9. SOLUTION_KEYWORD (Normalized symptom keywords for diagnostic matching)
10. APPOINTMENT (Service booking and completed maintenance log)
11. INVOICE (Itemized billing and payment status)
12. CONVERSATION (Single-table message repository for appointment chat)

18. Recommended Entity Structure

- **WORKSHOP (Strong):** The tenant entity; holds name, address, and unique access code.
- **USER (Strong / Supertype):** Base credentials, workshop foreign key, and role enumeration (ADMIN, CUSTOMER, MECHANIC).
- **CUSTOMER (Subtype of USER):** Contains customer-specific contact information (phone).
- **MECHANIC (Subtype of USER):** Contains mechanic-specific staff identification (employee_code).
- **VEHICLE (Strong):** Vehicle identity and odometer records linked to an owner USER.
- **REMINDER (Strong):** Automated service reminder linked to a VEHICLE.
- **PROBLEM_REPORT (Strong):** Raw customer symptom log linked to a VEHICLE and reporting USER.
- **SOLUTION_REPORT (Strong / Child of Problem Report):** AI-generated analysis (probable cause, recommended action, urgency, confidence score) linked 1:1 to a PROBLEM_REPORT.
- **SOLUTION_KEYWORD (Weak / Associative):** Atomic keywords linked to a SOLUTION_REPORT via composite PK (solution_id, symptom_keyword).
- **APPOINTMENT (Strong):** Service reservation and post-service log (work details, parts cost, labor cost) linked to VEHICLE, mechanic USER, and optional PROBLEM_REPORT.
- **INVOICE (Strong / Child of Appointment):** Billing record linked 1:1 to APPOINTMENT.
- **CONVERSATION (Strong):** Individual chat messages linked to APPOINTMENT and sender USER.

19. Entity Details (Attributes & Data Types)

WORKSHOP

```
workshop_id INT AUTO_INCREMENT (PK)
name VARCHAR(100) NOT NULL
address VARCHAR(200)
access_code VARCHAR(50) UNIQUE NOT NULL
```

USER

```
user_id INT AUTO_INCREMENT (PK)
workshop_id INT NOT NULL (FK -> WORKSHOP.workshop_id)
first_name VARCHAR(50) NOT NULL
last_name VARCHAR(50) NOT NULL
email VARCHAR(100) UNIQUE NOT NULL
password_hash VARCHAR(255) NOT NULL
role ENUM('ADMIN', 'CUSTOMER', 'MECHANIC') NOT NULL
```

CUSTOMER

```
user_id INT (PK, FK -> USER.user_id ON DELETE CASCADE)
phone VARCHAR(20)
```

MECHANIC

```
user_id INT (PK, FK -> USER.user_id ON DELETE CASCADE)
employee_code VARCHAR(20) UNIQUE NOT NULL
```

VEHICLE

```
vehicle_id INT AUTO_INCREMENT (PK)
owner_id INT NOT NULL (FK -> USER.user_id)
vin VARCHAR(17) UNIQUE NOT NULL
make VARCHAR(50) NOT NULL
model VARCHAR(50) NOT NULL
year INT NOT NULL
odometer INT NOT NULL
```

REMINDER

```
reminder_id INT AUTO_INCREMENT (PK)
vehicle_id INT NOT NULL (FK -> VEHICLE.vehicle_id ON DELETE CASCADE)
reminder_type VARCHAR(50) NOT NULL
due_date DATE NOT NULL
message TEXT
status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
```

PROBLEM_REPORT

```
report_id INT AUTO_INCREMENT (PK)
customer_id INT NOT NULL (FK -> USER.user_id)
vehicle_id INT NOT NULL (FK -> VEHICLE.vehicle_id ON DELETE CASCADE)
description TEXT NOT NULL
status VARCHAR(20) NOT NULL DEFAULT 'OPEN'
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
```

SOLUTION_REPORT

```
solution_id INT AUTO_INCREMENT (PK)
report_id INT NOT NULL UNIQUE (FK -> PROBLEM_REPORT.report_id ON DELETE CASCADE)
description TEXT
probable_cause TEXT NOT NULL
recommended_action TEXT NOT NULL
urgency VARCHAR(10) NOT NULL
confidence_score DECIMAL(4,3)
reviewed_by INT (FK -> USER.user_id ON DELETE SET NULL)
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
```

```

SOLUTION_KEYWORD
solution_id INT NOT NULL (PK, FK -> SOLUTION_REPORT.solution_id ON DELETE CASCADE)
symptom_keyword VARCHAR(50) NOT NULL (PK)

APPOINTMENT
appointment_id INT AUTO_INCREMENT (PK)
vehicle_id INT NOT NULL (FK -> VEHICLE.vehicle_id)
mechanic_id INT NOT NULL (FK -> USER.user_id)
report_id INT NULL (FK -> PROBLEM_REPORT.report_id ON DELETE SET NULL)
scheduled_start DATETIME NOT NULL
duration_minutes INT NOT NULL
status VARCHAR(20) NOT NULL DEFAULT 'SCHEDULED'
service_description TEXT
parts_cost DECIMAL(10,2) DEFAULT 0.00
labor_cost DECIMAL(10,2) DEFAULT 0.00

INVOICE
invoice_id INT AUTO_INCREMENT (PK)
appointment_id INT NOT NULL UNIQUE (FK -> APPOINTMENT.appointment_id ON DELETE CASCADE)
total_amount DECIMAL(10,2) NOT NULL
status VARCHAR(20) NOT NULL DEFAULT 'PENDING'
issued_at DATETIME DEFAULT CURRENT_TIMESTAMP

CONVERSATION
conversation_id INT AUTO_INCREMENT (PK)
appointment_id INT NOT NULL (FK -> APPOINTMENT.appointment_id ON DELETE CASCADE)
sender_id INT NOT NULL (FK -> USER.user_id)
content TEXT NOT NULL
sent_at DATETIME DEFAULT CURRENT_TIMESTAMP

```

20. Composite Attributes

Address fields are modeled as atomic text strings or broken into standard atomic fields (address) within WORKSHOP. User names are strictly decomposed into first_name and last_name.

21. Multivalued Attributes

Symptom keywords for AI solutions are normalized into individual rows within SOLUTION_KEYWORD (satisfying 1NF). User phone numbers are stored as atomic single attributes in CUSTOMER.

22. Derived Attributes

total_amount in INVOICE is computed at appointment completion from parts_cost + labor_cost and stored to preserve billing history. full_name is dynamically concatenated in the application layer and not stored. vehicle_age is calculated from YEAR(CURDATE()) - year and omitted from the database schema.

23. Primary Keys / Candidate Keys / Composite Keys

- **Primary Keys:** Surrogate auto-increment keys for all primary entities.
- **Composite Primary Key:** (solution_id, symptom_keyword) on SOLUTION_KEYWORD.
- **Candidate / Unique Keys:** WORKSHOP.access_code, USER.email, MECHANIC.employee_code, VEHICLE.vin, SOLUTION_REPORT.report_id (1:1), INVOICE.appointment_id (1:1).

24. Foreign Keys & Integrity Constraints

- USER.workshop_id → WORKSHOP.workshop_id (ON DELETE RESTRICT)
- CUSTOMER.user_id → USER.user_id (ON DELETE CASCADE)

- MECHANIC.user_id → USER.user_id (ON DELETE CASCADE)
- VEHICLE.owner_id → USER.user_id (ON DELETE RESTRICT)
- REMINDER.vehicle_id → VEHICLE.vehicle_id (ON DELETE CASCADE)
- PROBLEM_REPORT.customer_id → USER.user_id (ON DELETE RESTRICT)
- PROBLEM_REPORT.vehicle_id → VEHICLE.vehicle_id (ON DELETE CASCADE)
- SOLUTION_REPORT.report_id → PROBLEM_REPORT.report_id (ON DELETE CASCADE)
- SOLUTION_REPORT.reviewed_by → USER.user_id (ON DELETE SET NULL)
- SOLUTION_KEYWORD.solution_id → SOLUTION_REPORT.solution_id (ON DELETE CASCADE)
- APPOINTMENT.vehicle_id → VEHICLE.vehicle_id (ON DELETE RESTRICT)
- APPOINTMENT.mechanic_id → USER.user_id (ON DELETE RESTRICT)
- APPOINTMENT.report_id → PROBLEM_REPORT.report_id (ON DELETE SET NULL)
- INVOICE.appointment_id → APPOINTMENT.appointment_id (ON DELETE CASCADE)
- CONVERSATION.appointment_id → APPOINTMENT.appointment_id (ON DELETE CASCADE)
- CONVERSATION.sender_id → USER.user_id (ON DELETE RESTRICT)

25. Identifying Relationships (Weak Entities)

SOLUTION_KEYWORD is an identifying weak entity dependent on SOLUTION_REPORT, utilizing solution_id as part of its composite primary key. CUSTOMER and MECHANIC are subtype entities that share their primary key with USER.user_id.

26. Recursive Relationships

No recursive unary relationships exist in this schema version.

27. Generalization / Specialization (ISA)

An EER supertype/subtype hierarchy is implemented: USER serves as the supertype containing shared authentication attributes (email, password_hash, first_name, last_name, role, workshop_id). CUSTOMER is a specialization containing customer-specific fields (phone). MECHANIC is a specialization containing staff-specific fields (employee_code).

28. Relationship & Cardinality Table

Entity A	Entity B	Cardinality	Participation	Description	Associative Entity?
WORKSHOP	USER	1:N	Workshop (1,1), User (1,1)	Workshop employs staff and onboards customers	No
USER	CUSTOMER	1:1	User (0,1), Customer (1,1)	Customer subtype specialization	No
USER	MECHANIC	1:1	User (0,1), Mechanic (1,1)	Mechanic subtype specialization	No
USER (Customer)	VEHICLE	1:N	User (1,1), Vehicle (1,1)	Customer owns one or more vehicles	No
VEHICLE	REMINDER	1:N	Vehicle (1,1), Reminder (0,*)	Vehicle has scheduled service reminders	No
VEHICLE	PROBLEM_REPORT	1:N	Vehicle (1,1), Report (0,*)	Issues reported against a vehicle	No
PROBLEM_REPORT	SOLUTION_REPORT	1:1	Report (1,1), Solution (0,1)	AI-generated diagnostic report for an issue	No
SOLUTION_REPORT	SOLUTION_KEYWORD	1:N	Solution (1,1), Keyword (1,*)	Normalized search keywords for solution	Yes (Identifying)
VEHICLE	APPOINTMENT	1:N	Vehicle (1,1), Appt (0,*)	Vehicle booked for maintenance	No
USER (Mechanic)	APPOINTMENT	1:N	User (1,1), Appt (0,*)	Mechanic assigned to service appointment	No
APPOINTMENT	INVOICE	1:1	Appt (1,1), Invoice (0,1)	Invoice generated upon service completion	No
APPOINTMENT	CONVERSATION	1:N	Appt (1,1), Message (0,*)	Messages exchanged regarding appointment	No

29. Tenant Ownership / Isolation Map

Multi-tenancy is enforced through the root foreign key USER.workshop_id. No child tables store redundant workshop_id columns, eliminating transitive functional dependencies and ensuring 3NF compliance. Tenant data isolation is enforced at runtime by resolving the authenticated user's workshop_id in the application service layer and joining across direct relational foreign keys.

30. Appointment & Concurrency Modeling

APPOINTMENT stores scheduled_start (DATETIME) and duration_minutes (INT). Database unique constraints on (mechanic_id, scheduled_start) cannot prevent overlapping ranges. Concurrency is enforced via transactional checks using pessimistic locking (SELECT ... FOR UPDATE):

```
SELECT COUNT(*) FROM APPOINTMENT
WHERE mechanic_id = :mechanicId
  AND status != 'CANCELLED'
  AND scheduled_start < :newEndTime
  AND DATE_ADD(scheduled_start, INTERVAL duration_minutes MINUTE) > :newStartTime;
```

If the count exceeds zero, the transaction rolls back and the booking request is rejected.

31. AI / RAG Architecture

Customer logs a problem in PROBLEM_REPORT. Backend extracts symptom tokens, retrieves matching procedural templates from internal static catalogs, and constructs a structured prompt. Google Gemini LLM processes the prompt and returns a structured JSON payload containing: probable_cause, recommended_action, urgency, and confidence_score. The backend persists the output in SOLUTION_REPORT and inserts associated search tokens into SOLUTION_KEYWORD.

32. Event-Driven Architecture

In-process Node.js EventEmitter manage domain side-effects without table bloat: ProblemReportedEvent (dispatches async AI diagnostic processing), AppointmentBookedEvent (sends booking confirmations), and ServiceCompletedEvent (automatically triggers INVOICE record creation).

33. Backend Architecture

- **Route / Controller Layer:** Exposes REST endpoints (Express router), validates request payloads/DTOs, and handles HTTP response mapping.
- **Authentication / Authorization Middleware:** Intercepts requests, validates JWT claims, and injects authenticated user context (user_id, workshop_id, role).
- **Service Layer:** Executes transactional business logic, coordinates LLM API calls, and enforces concurrency locks via explicit SQL transaction management.
- **Data Access / Repository Layer:** Raw SQL repository modules executing parameterized SQL statements directly via mysql2 connection pool (without ORM).

34. Frontend Architecture

Workshop Dashboard (admin portal), Vehicle Portal (customer garage view), Diagnostic Console (issue submission & solution report viewer), Booking Scheduler (conflict-aware slot calendar), and Appointment Hub (integrated view containing appointment status, service history details, invoice summaries, and chat interface).

35. Phase-by-Phase Implementation Workflow

- **Phase 0 – Planning & Architecture (Week 1):** Finalize 3NF schema, approve EER specialization, and configure repository.
- **Phase 1 – Database Deployment (Week 2):** Execute DDL scripts, configure foreign key constraints, insert seed data, and verify relational joins.

- **Phase 2 – Core Backend & Security (Week 3):** Bootstrap Node.js / Express.js, configure JWT authentication middleware & BCrypt password hashing, establish mysql2 connection pool, and implement raw SQL user/auth repository modules.
- **Phase 3 – Vehicle & Issue Subsystem (Week 4):** Build vehicle management APIs, problem report endpoints, and reminder cron jobs.
- **Phase 4 – AI Diagnostic Integration (Week 5):** Integrate Gemini REST API, build JSON parser, and implement SOLUTION_REPORT and SOLUTION_KEYWORD persistence.
- **Phase 5 – Appointment & Concurrency Engine (Week 6):** Build booking engine with pessimistic range-locking transactions (SELECT ... FOR UPDATE) and mechanic slot calculators.
- **Phase 6 – Messaging & Billing Subsystem (Week 7):** Implement single-table CONVERSATION chat APIs and automatic INVOICE generation upon appointment completion.
- **Phase 7 – Frontend Assembly (Week 8):** Build Next.js dashboards, booking UI, chat components, and vehicle health views.
- **Phase 8 – Testing, Validation & CI/CD (Week 9):** Execute parallel concurrency tests, mock AI responses, build Docker Compose manifests, and deploy via GitHub Actions.

36. Testing Strategy

- **Unit Testing:** Jest / Vitest and Supertest for service-layer business rules, controllers, and input validation.
- **Database Integration Testing:** Integration test suite executing against MySQL 8.0 instance to validate raw SQL queries, foreign key cascades, constraints, and ACID transactions.
- **Concurrency Testing:** Multi-threaded / concurrent async worker tests simulating simultaneous booking requests for identical mechanic slots to verify transactional locking.
- **AI Mock Testing:** Mock simulations of Gemini API payloads to verify robust error handling and parsing.

37. Docker & Deployment Configuration

docker-compose.yml orchestrates: mysql-db (MySQL 8.0 container with persistent health checks), backend-api (Node.js/Express.js container), and frontend-ui (Next.js production build container).

38. CI/CD Pipeline

GitHub Actions workflow triggers on push to main: lint and compile Next.js frontend, run backend test suite against containerized MySQL, and build/verify Docker images.

39. Monitoring & Logging

Express health probe endpoints (/api/health, /api/ready). Structured JSON logging (via Winston / Pino) capturing appointment events, authentication attempts, database query execution, and AI execution latency.

40. Documentation Deliverables

- Complete MySQL DDL scripts with constraints and indexes.
- OpenAPI / Swagger REST API specifications.
- EER schema diagram illustrating supertype/subtype hierarchy and cardinalities.
- 3NF Normalization mathematical proofs.

41. Timeline / Milestones

```
+-----+
| W1: DB Design -> W2: Schema DDL -> W3: Auth / Raw SQL -> W4: Vehicles / Reports |
| W5: AI Diagnostics -> W6: Concurrency Booking -> W7: Chat & Invoicing |
| W8: Frontend UI -> W9: Testing, Docker & Final Release |
+-----+
```

42. Final Deliverables

- Production-grade MySQL DDL script implementing the 11-table schema.
- Express.js (Node.js) backend source code with Raw SQL repositories and concurrency controls.
- Next.js frontend application with Tailwind UI components.
- Docker Compose configuration for one-command local orchestration.
- Comprehensive automated test suite covering concurrency, security, and schema integrity.

43. Final Architecture Summary

AutoCare AI (V1) is engineered around a clean, 3NF-compliant relational schema consisting of 11 core tables. The design decouples diagnostic outputs into SOLUTION_REPORT, normalizes keywords into SOLUTION_KEYWORD, optimizes real-time messaging into a single CONVERSATION table, and models user roles via clean supertype/subtype specialization (USER, CUSTOMER, MECHANIC). Multi-tenancy is enforced through the root user workshop link without transitive key duplication, while appointment booking integrity is protected via transactional range locking.

Itemized Change Ledger (Modifications Applied)

- **Separated AI Diagnostic Outputs:** Removed probable_cause, urgency, recommended_action, and confidence_score from PROBLEM_REPORT. Created SOLUTION_REPORT linked 1:1 with PROBLEM_REPORT and added reviewed_by tracking.
- **Normalized Symptom Keywords:** Added the SOLUTION_KEYWORD composite-key table (solution_id, symptom_keyword) to satisfy 1NF.
- **Streamlined Chat Architecture:** Replaced the two-table Conversation and Message model with a single CONVERSATION table where each row stores an individual message linked to appointment_id.
- **Implemented Role Subtyping:** Updated USER into a supertype entity with CUSTOMER (storing phone) and MECHANIC (storing employee_code) subtype tables.
- **Refactored Multi-Tenancy:** Removed redundant workshop_id foreign keys from downstream entities (VEHICLE, APPOINTMENT, PROBLEM_REPORT) to eliminate transitive dependencies and ensure 3NF compliance.
- **Updated Concurrency Strategy:** Replaced simple unique index assumptions with transactional range checks (SELECT ... FOR UPDATE) to prevent overlapping appointment intervals.
- **Normalized Service Details:** Added service_description, parts_cost, and labor_cost directly to APPOINTMENT and eliminated redundant stored derived totals.
- **Integrated Reminders Entity:** Formalized the REMINDER table to support predictive maintenance schedules.
- **Shifted to Pure Raw SQL (Non-ORM) Architecture:** Replaced Spring Data JPA/Hibernate ORM with a Node.js/Express.js backend utilizing direct mysql2 driver parameterized queries and explicit SQL transaction management, fully adhering to the strict requirement of zero ORM usage.